gemstone-rs Release Checklist

Crates, VSIX, GitHub release, and verification steps

Generated from the Markdown sources in gemstone-rs/docs.

Contents

Release Checklist

Use this checklist for coordinated crate, VSIX, and GitHub releases.

0.2.1 / Workbench 0.3.1 Notes

- CLI setup: global `--env-file`, `doctor --strict`, and safer `.env.gemstone-rs` template loading across CLI and explorer startup.
- Codegen: `codegen explain --json`, schema files, and a generated-wrapper compile smoke test.
- Mapping: missing keys, nested dictionaries, and array reads now report richer path context such as `booking.items[2]`.
- Explorer: setup assistant, env sample/write, and a structured Codegen Explain action.
- VS Code: `GemStone RS: Verify Strict Setup` and automatic env-file passing when `gemstoneRs.envFile` exists.

Workbench 0.3.2 Notes

- VS Code: `GemStone RS: Run Setup Assistant` renders the explorer `/api/setup/assistant` checks in the output panel.
- CI: schema copies and `codegen explain --json` output are validated by `scripts/validate_codegen_schemas.js`.
- Examples: the generated-wrapper smoke crate now runs `cargo test`.

Workbench 0.3.3 Notes

- CLI: `codegen explain-profile [--json] <profile-name> [profile-file]` resolves project profiles before rendering codegen summaries.

- VS Code: `GemStone RS: Codegen Explain` and `GemStone RS: Codegen Explain Profile` render structured classes, selectors, mappings, and test stubs.

0.2.2 / Workbench 0.3.4 Notes

- Core: project profile check reports now live in `gemstone-rs` and are shared by CLI, explorer, and VS Code.
- Explorer: `/api/codegen/profiles/check` returns the shared JSON report and the browser renders a profile status table with Preview, Diff, Check, and Generate actions.
- Tooling: `schemas/gemstone-rs.profile-check.schema.json` documents the shared report shape, and CI runs a real explorer HTTP endpoint smoke test.

Before Release

- Update crate versions in `Cargo.toml` files.
- Update `vscode-gemstone-rs-workbench/package.json`.
- Regenerate codegen examples:

```
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check-profile default examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile list --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile show default --json examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile resolve default --json examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- profile check --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile sample > /tmp/gemstone-rs.codegen-
profiles.json
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
```

```
diff -u examples/codegen/gemstone-rs.codegen-profiles.json /tmp/gemstone-rs.codegen-
profiles.json
```

- Refresh screenshots when the explorer or workbench UI changed:

```
make screenshots
```

- Run local verification:

```
python3 scripts/version_check.py
make verify
make vscode-package
python3 docs/build_pdf_docs.py
```

Or use the dry-run release wrapper:

```
DRY_RUN=1 scripts/release_all.sh 0.2.2
```

`make verify` includes the version check and checks that PDF generation completes and produces non-empty PDF files. The release wrapper writes repository-relative SHA256 entries and verifies the expected VSIX plus every PDF with `scripts/verify_release_artifacts.py`. The release workflow rebuilds and attaches fresh PDFs for the target runner because WeasyPrint output can differ byte-for-byte across platforms. The VSIX filename uses `vscode-gemstone-rs-workbench/package.json`; the crate release tag still uses the workflow `version` input.

Commit Review Pass

Before publishing, make a deliberate release commit review pass:

```
git status --short
git diff --stat
make verify
```

Confirm the release commit includes any regenerated PDFs, refreshed screenshots, profile schema changes, sample profile updates, and VSIX docs. Keep publishing for a separate step unless the release workflow is intentionally being run from that commit.

Publish

Set GitHub secrets once:

```
gh secret set CARGO_REGISTRY_TOKEN
gh secret set VSCE_PAT
```

Run the release workflow:

```
gh workflow run release.yml \
  --ref main \
  -f version=0.2.2 \
  -f publish-crates=true \
  -f publish-vsix=true \
  -f create-github-release=true \
  -f dry-run=false
```

The workflow publishes crates in dependency order:

1. `gemstone-gci`
2. `gemstone-rs-macros`
3. `gemstone-rs`
4. `gemstone-rs-cli`
5. `gemstone-rs-explorer`

It also packages/publishes the VSIX and can create the GitHub release with the VSIX attached.

Manual crate publishing remains available:

```
scripts/publish_crates.sh
```

For a dry run:

```
DRY_RUN=1 scripts/publish_crates.sh
```

Manual end-to-end publishing remains available through the release wrapper:

```
PUBLISH_CRATES=1 PUBLISH_VSIX=1 CREATE_GITHUB_RELEASE=1 VERIFY_PUBLIC=1 DRY_RUN=0
scripts/release_all.sh 0.2.2
```

The GitHub `Release` workflow now calls the same `scripts/release_all.sh` wrapper, so local and Actions releases share the same verification, packaging, checksum, publish, GitHub release, and post-publish verification path.

Dry-run mode checks each crate's publish file list locally. The real publish path still uses `cargo publish` in dependency order so crates.io resolves each new dependency after it has been published. If a workflow is rerun after a partial publish, already-published crate versions are skipped and the remaining crates continue.

Post-Release Verification

```
scripts/publish_verify.sh 0.2.2
```

The script checks:

- crates.io package/version JSON for all crates
- `cargo install gemstone-rs-cli`
- `cargo install gemstone-rs-explorer`
- `gemstone-rs --help`
- `gemstone-rs-explorer --help`
- VS Code Marketplace version matches `package.json`
- GitHub Release `v<version>` exists
- GitHub Release assets include the VSIX, `SHA256SUMS`, and every generated PDF
- downloaded GitHub Release assets match the published `SHA256SUMS`

To skip the GitHub Release asset check during early testing:

```
VERIFY_GITHUB_RELEASE=0 scripts/publish_verify.sh 0.2.2
```

Optional Live Smoke

Run the manual GitHub Actions live job with GemStone secrets configured, or run locally:

```
GS_RUN_LIVE_RUST=1 cargo test -p gemstone-rs live_ -- --test-threads=1
```

The live smoke coverage includes login/logout, `3 + 4 == 7`, global put/get, string round-trip, `perform`, commit, abort, browser lookup of `Object`, and generated wrapper `printString`.

The `--test-threads=1` flag is intentional. GemStone GCI has process-global session state, so live tests should not be run concurrently.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.